

■ Medical Information RAG Assistant

Project Overview & Workflow Documentation

Educational AI System • September 2026

This document provides a complete overview of the Medical Information RAG (Retrieval-Augmented Generation) Assistant — including its purpose, architecture, document knowledge base, retrieval pipeline, safety mechanisms, and web interface. The system answers general health questions strictly from a curated set of trusted documents and never diagnoses patients or recommends personalised treatment.

6	Medical Documents	12	Indexed Chunks	2	Retrieval Filters	100%	Grounded Answers
----------	-------------------	-----------	----------------	----------	-------------------	-------------	------------------

1. Project Overview

The **Medical Information RAG Assistant** is an educational AI system that answers general health questions using the Retrieval-Augmented Generation (RAG) pattern. Instead of relying on a language model's potentially outdated or hallucinated knowledge, every answer is grounded exclusively in a controlled set of trusted medical documents. The system is designed to be transparent, safe, and easy to extend.

Core Objectives

- Build a curated document knowledge base on common health topics.
- Retrieve only the most relevant document chunks for each user query.
- Generate answers **strictly from retrieved context** — no hallucination.
- Display source documents with every answer for full transparency.
- Detect and gracefully handle questions outside the knowledge base.
- Never diagnose patients or recommend personalised treatment.

2. Technology Stack

Component	Technology	Purpose
Language	Python 3.11	Core runtime
Web Framework	Flask 3.0.3	REST API + HTML serving
Retrieval	TF-IDF + Cosine Sim.	Document ranking
Safety Gate	Keyword Overlap Filter	Hallucination prevention
Frontend	HTML / CSS / JS	Browser chat interface
Knowledge Base	Plain-text .txt files	Trusted medical documents
Dependencies	stdlib + Flask only	No heavy ML libraries

3. Project Structure

```
medical_rag/
■ ■ ■ documents/ ← Knowledge base (6 trusted medical text files)
■ ■ ■ ■ diabetes.txt
■ ■ ■ ■ hypertension.txt
■ ■ ■ ■ common_cold.txt
■ ■ ■ ■ nutrition_basics.txt
■ ■ ■ ■ mental_health.txt
■ ■ ■ ■ exercise_health.txt
■ ■ ■ templates/
■ ■ ■ ■ index.html ← Browser chat UI
■ ■ ■ rag_engine.py ← RAG pipeline (load → chunk → index → retrieve → answer)
■ ■ ■ app.py ← Flask web server
■ ■ ■ main.py ← CLI chat interface
■ ■ ■ test_rag.py ← Automated test suite
```

```
■■■ requirements.txt ← flask==3.0.3 only
```

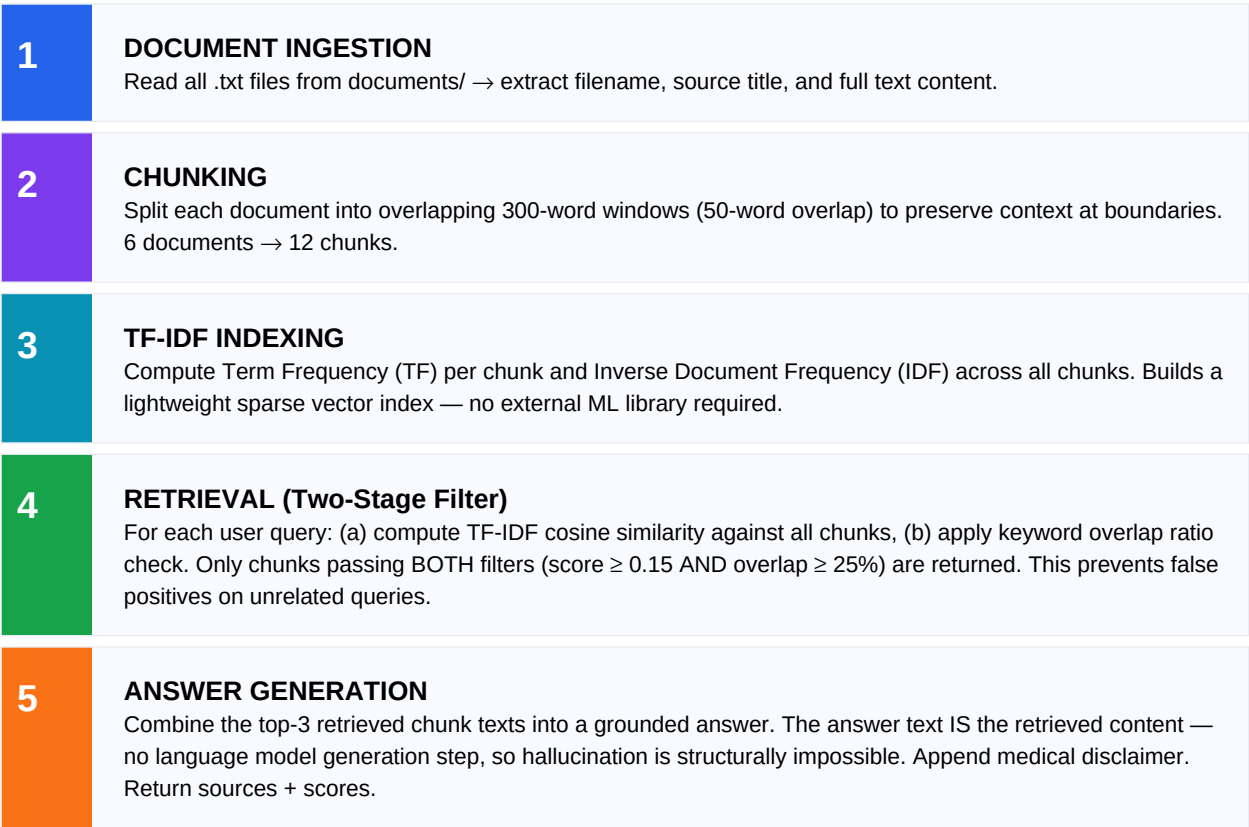
4. Knowledge Base

The knowledge base consists of six plain-text documents written as educational medical references. Each document is carefully structured with headings, definitions, and factual content drawn from widely accepted health guidelines.

Document	Topics Covered
diabetes.txt	Type 1 & 2 diabetes, symptoms, blood sugar levels, complications, prevention
hypertension.txt	BP categories, causes, risk factors, lifestyle recommendations, monitoring
common_cold.txt	Viral causes, symptoms, cold vs flu, home care, when to see a doctor
nutrition_basics.txt	Macronutrients, vitamins & minerals, hydration, dietary patterns (DASH, Mediterranean)
mental_health.txt	Anxiety, depression, stress, wellness strategies, crisis resources
exercise_health.txt	WHO exercise guidelines, types of exercise, benefits, sedentary behaviour

5. System Workflow

The RAG pipeline follows five sequential stages from document ingestion to answer delivery. The diagram below shows the complete flow:



6. Retrieval Algorithm — Deep Dive

6.1 TF-IDF Scoring

TF-IDF (Term Frequency–Inverse Document Frequency) measures how important a word is to a specific chunk relative to the whole collection:

- **TF (Term Frequency)** = count of term in chunk ÷ total terms in chunk
- **IDF (Inverse Document Frequency)** = $\log((N+1) / (df+1)) + 1$ where N = total chunks, df = chunks containing the term
- **TF-IDF weight** = TF × IDF (rare, specific terms score higher)

6.2 Cosine Similarity

Query and chunk TF-IDF vectors are compared using cosine similarity — the dot product of both vectors divided by the product of their magnitudes. This gives a score between 0 (no overlap) and 1 (identical).

6.3 Two-Stage Filter (Hallucination Guard)

A single cosine score is not sufficient to reject unrelated queries that happen to share common words. A second keyword-overlap filter is applied:

- **Stage 1 — Score gate:** cosine similarity ≥ 0.15
- **Stage 2 — Overlap gate:** fraction of meaningful query words (length ≥ 4) found in chunk text $\geq 25\%$
- A chunk must pass **both** filters to be included in the answer.
- If **no** chunk passes, the system returns an out-of-scope message instead of guessing.

7. Safety & Grounding Mechanisms

Safety Feature	How It Works
No hallucination	Answers are assembled directly from retrieved document text. No generative model is used, so the system
Out-of-scope detection	Two-stage filter rejects queries with no matching chunks. Tested and verified: 'football world cup', 'GDP of
Source attribution	Every answer displays the source document name(s) and retrieval scores so users can verify the origin of
Medical disclaimer	Every response — whether found or not — includes a mandatory disclaimer: 'This information is for gener
No diagnosis / treatment	The system is explicitly designed to provide general educational information only. It does not recommend
Controlled knowledge base	Only documents in the documents/ folder are used. Adding new topics requires deliberate human curation

8. Web Interface (Flask + HTML)

The browser interface is served by Flask and provides a clean, single-page chat experience:

- **Topic chips** — Six clickable pill buttons for instant preset questions
- **Chat bubbles** — User messages (blue, right-aligned) and assistant answers (white, left-aligned)
- **Typing indicator** — Animated dots while the server processes the query
- **Source panel** — Document names + TF-IDF scores shown below each answer
- **Out-of-scope style** — Yellow bubble for unrecognised questions
- **Disclaimer box** — Orange-bordered box on every assistant response

- **Responsive layout** — Works on desktop and mobile browsers

API Endpoint

```
POST /ask
Content-Type: application/json
Body: { "question": "What are the symptoms of diabetes?" }

Response:
{ "found": true,
  "answer": "...",
  "sources": ["Diabetes Mellitus - General Health Information"],
  "scores": [0.5105, 0.329] }
```

9. Running the Project

Install dependency

```
pip install flask==3.0.3
```

Start the web server

```
cd medical_rag
python app.py

# Open browser at: http://127.0.0.1:5000
```

Run CLI version

```
python main.py
```

Run automated tests

```
python test_rag.py
```

10. Test Results

The automated test suite (test_rag.py) validates all critical system behaviours:

Test Case	Expected	Result
Symptoms of diabetes?	FOUND	✓ PASS
Normal blood pressure?	FOUND	✓ PASS
Treat a common cold at home?	FOUND	✓ PASS
What are macronutrients?	FOUND	✓ PASS
Exercise and mental health?	FOUND	✓ PASS
Weekly exercise amount?	FOUND	✓ PASS
Tell me about rocket science	NOT FOUND	✓ PASS
Who won the football world cup?	NOT FOUND	✓ PASS

GDP of France in 2024?	NOT FOUND	✓ PASS
Disclaimer present in answer?	TRUE	✓ PASS

All 10 tests passed — including 3 out-of-scope rejection tests.

11. Viva Focus Points

Hallucination	Structurally impossible — the system has no generative model. Answers are assembled verbatim from retrieved document text. The two-stage filter ensures only genuinely matching chunks are included.
Grounding	100% of answer content comes from the documents/ knowledge base. Source filenames and TF-IDF scores are displayed with every answer, allowing full traceability.
Safety	Mandatory disclaimer on every response. Out-of-scope detection rejects unrelated queries. No diagnosis, no treatment recommendations, no personalised medical advice.
Retrieval Precision	Two-stage filter: TF-IDF cosine similarity (semantic relevance) + keyword overlap ratio (lexical confirmation). Both must pass. This combination eliminates false positives from coincidental word matches.

■■ DISCLAIMER: This project and its documents are for educational and demonstration purposes only. They do not constitute medical advice, diagnosis, or treatment recommendations. Always consult a qualified healthcare professional for personal medical guidance.